

Leo Marcinkus
ECEN 3730 – PCB Design Manufacture
12/08/25

Board 4 Report

Objective:

The objective of designing Board 4, the *instrument droid*, is to complete a fully working custom Arduino-compatible shield. It should be able to measure the output resistance of various voltage sources, utilize the Arduino IDE as normal, and be compatible with a standard Arduino Uno R3 board. This design required creative implementation of various peripherals, making independent design decisions, and implementing good standard engineering practices like including test points and isolation switches. Multiple voltage source inputs are included such as a 2-pin terminal block, a power jack, and a USB Mini port. After soldering, the design is validated by uploading sketches to verify functionality of each component and confirming correct operation and by comparing recorded data results to lab 21/22's data.

Project Overview:

1. Design an Arduino compatible PCB instrument droid
2. ATmega 328 from previous board should be able to boot and upload code
3. The DAC/ADC/MOSFET circuit must function
4. Smart LEDs should be implemented and function correctly
5. A buzzer should be implemented and function correctly
6. A switch to change sense resistors should be included. (300mA to 3mA)
7. Test points and isolation switches should be included for good debugging practices
8. The 10-pin and 8-pin headers should have a 0.160" gap between. An 8-pin header should be included on the other side. There should also be a 1.900" gap between the two sides to ensure compatibility with the Arduino
9. Test sketches should be uploaded to ensure board functionality
10. All components must be soldered neatly and correctly as good practice

Summary:

The purpose of this project was to design, assemble, and validate a 4-layer Instrument Droid Shield capable of sourcing controlled current while monitoring voltage, current sense voltage, and system behavior with voltage sources. The shield interfaces with our created ATmega328 from board 3, running created Arduino IDE Code, and includes a DAC output, ADC input, current-sense resistors, MOSFET switching stage, RGB LED indication, and an audible buzzer.

The microcontroller successfully boots and responds to commands via the Arduino IDE and serial interface. New code was developed in addition to the lab 21 code to increment the buzzer in musical scale steps at each measurement, to change the smart LED color based on the ratio of V_{Load}/V_{Thev} , and visually output the measurement data on a small OLED screen. The screen outputs exactly what the serial monitor shows, the measurement index, load current, Thevenin voltage, load voltage, and Thevenin resistance. This confirmed that the DAC output, ADC measurement chain, and coding logic were all operational.

Measurements were recorded using three different power sources: a regulated 5 V Arduino supply, a 9 V wall adapter, and a waveform generator's 50 Ohm DC output. For each source, data was collected in both the 300 mA (10 Ω sense resistor) and 3 A (1 Ω sense resistor) modes. The DAC waveform and op-amp outputs were verified using an oscilloscope, while ADC values, Thevenin resistance, and current behavior were logged through the serial monitor.

The measured results from the assembled PCB closely matched the results obtained previously on the solderless breadboard prototype from Lab 21, validating both the schematic design and PCB implementation. The load current, Thevenin voltage, load voltage, and Thevenin resistance all scaled correctly with load and sense-resistor selection, confirming proper constant-

current operation. Since load current changed appropriately with the code and sense-resistor selection, MOSFET switching and closed-loop regulation were also verified through system-level behavior.

There was one minor design flaw that occurred during layout. I mirrored the OLED header pins in Altium. This meant the display now has to be mounted backward on the final board. While this does not affect electrical functionality, it is a mechanical/layout issue that would be corrected in a future revision. If I were to redesign the board, additional effort would be spent on cleaner routing, improved symmetry, and optimized trace organization.

Overall, the my instrument droid met all functional objectives. The ATmega328 still successfully boots and executes commands, the DAC produces a stable output, the ADC accurately tracks voltage changes, the constant-current circuit operates in both current ranges, and the MOSFET switches the load as intended. The strong agreement between this board and Lab 21's breadboard measurements confirms the validity of my final design.

Schematic and PCB:

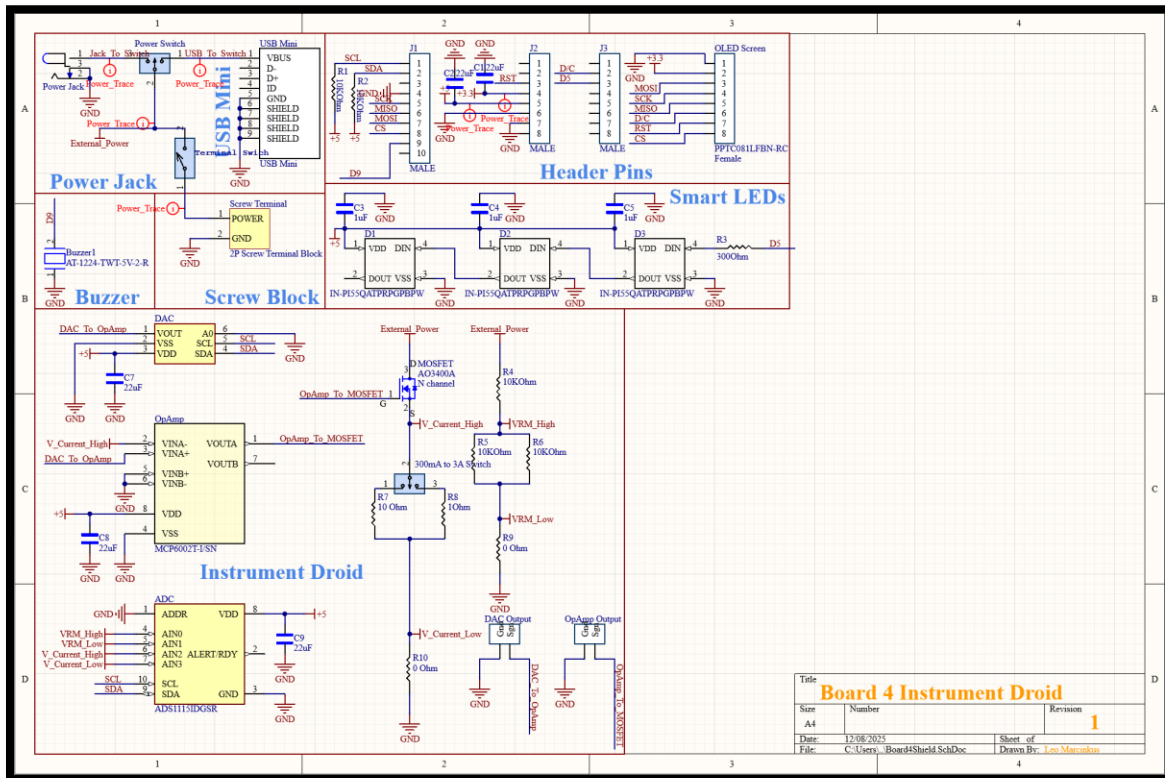


Figure 1: Altium Schematic

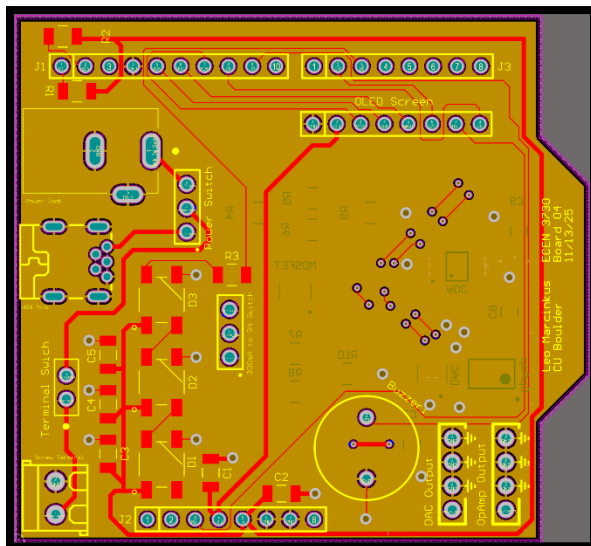


Figure 2: Altium PCB (Top of Board)

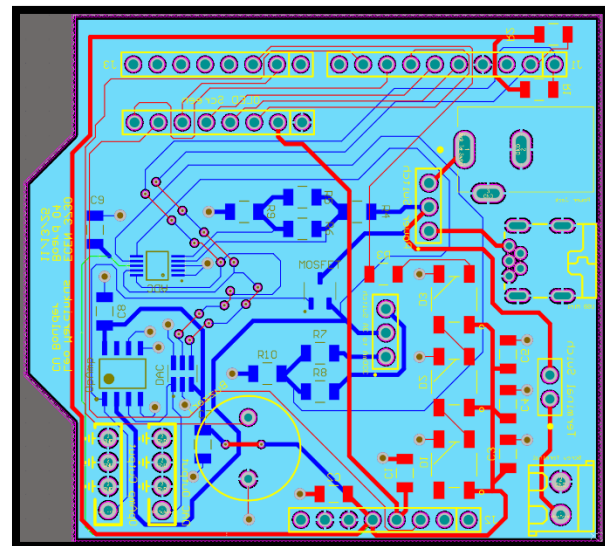


Figure 3: Altium PCB (Bottom of Board)

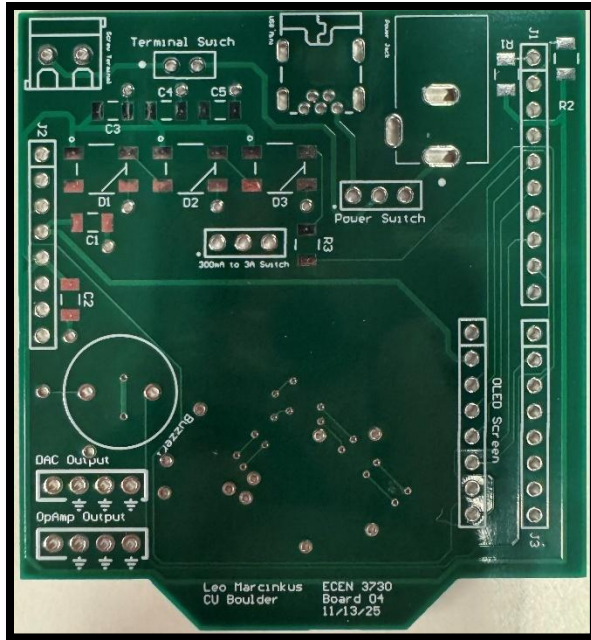


Figure 3: Unsoldered (Top of Board)

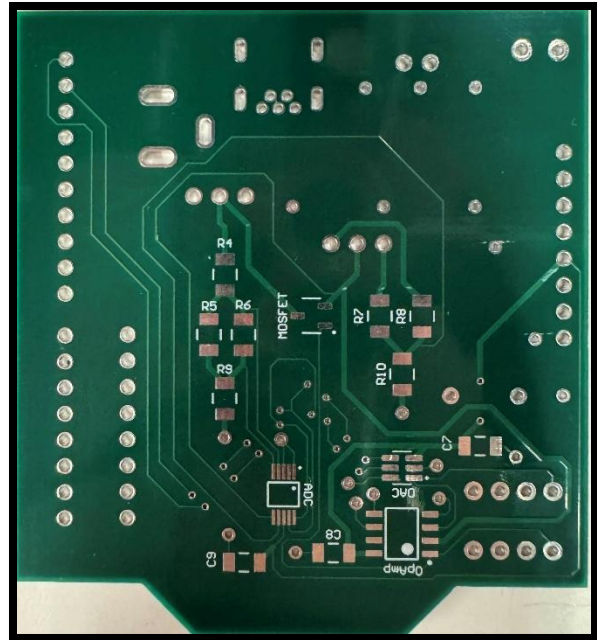


Figure 4: Unsoldered (Bottom of Board)

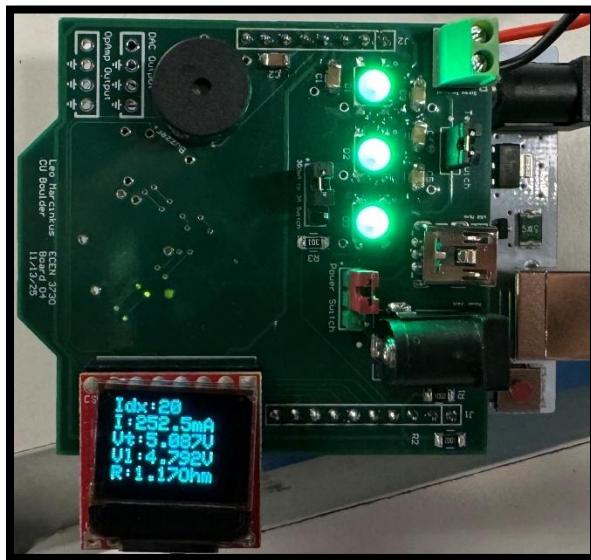


Figure 5: Soldered and Functioning (EX. 1)

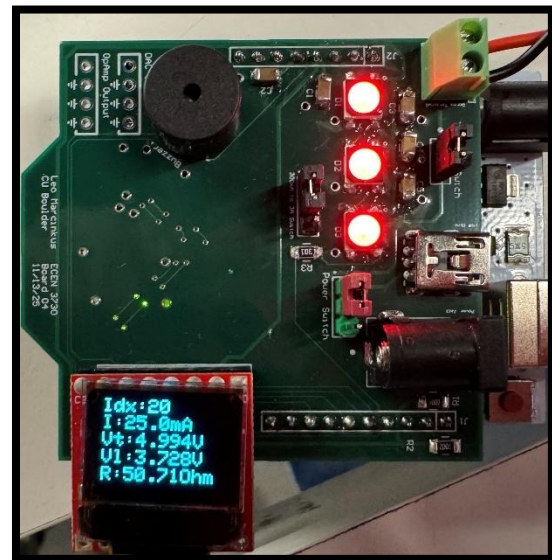
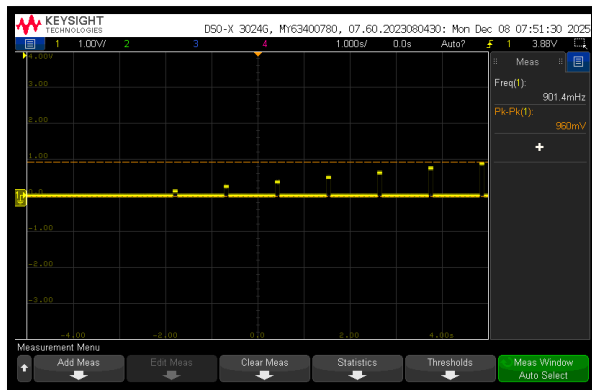
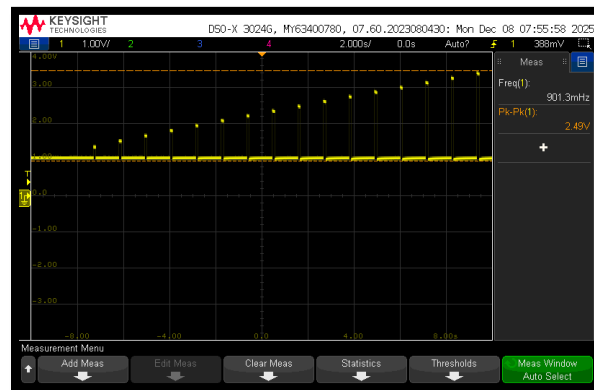
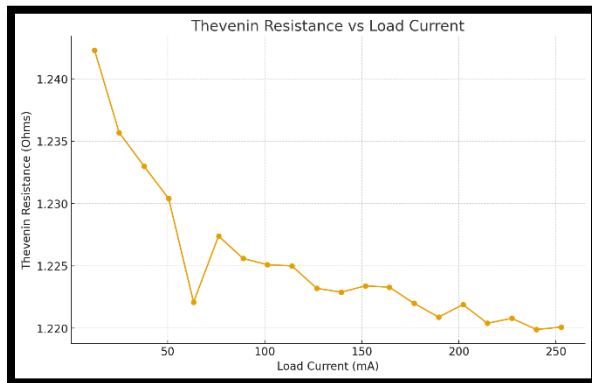
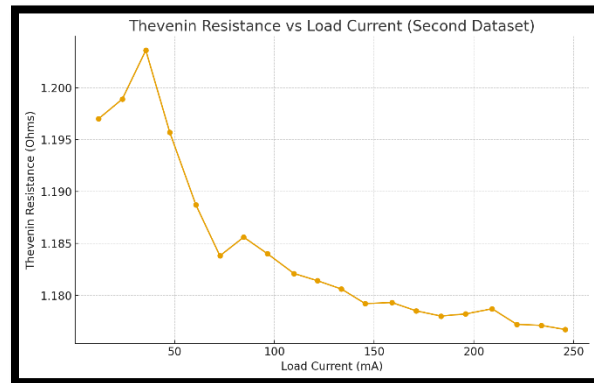
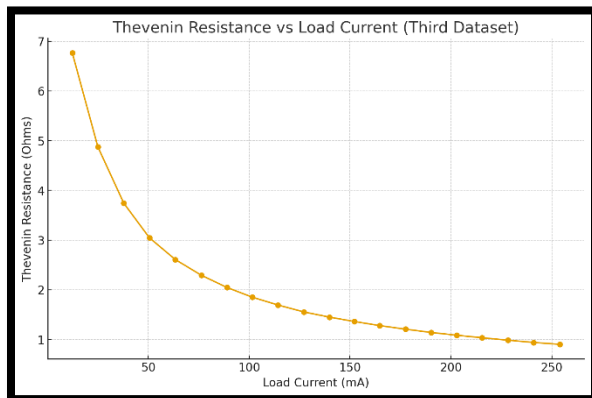
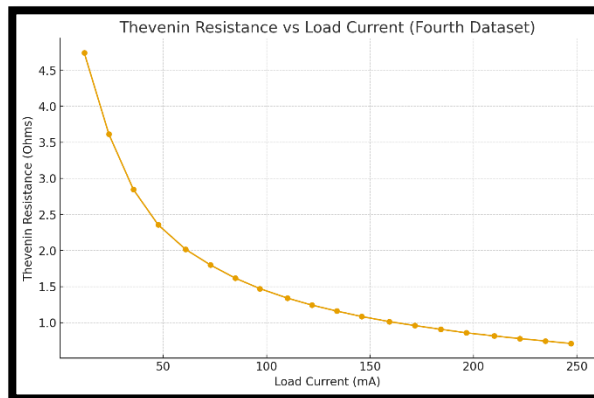


Figure 6: Soldered and Functioning (EX. 2)

Measurements:**DAC and OpAmp Output****DAC Output***Figure 7: DAC Output***OpAmp Output***Figure 8: OpAmp Output***Measurements (Recorded from Serial Monitor)****300 mA Switch***Figure 9: 5V Arduino Source***3A Switch***Figure 10: 5V Arduino Source**Figure 11: 9V Wall Source**Figure 12: 9V Wall Source*

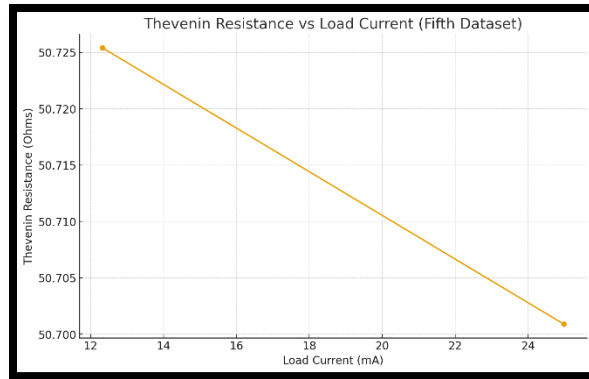


Figure 13: 50 Ω DC waveform

Compared to Lab 21's Data (Breadboard Prototype):

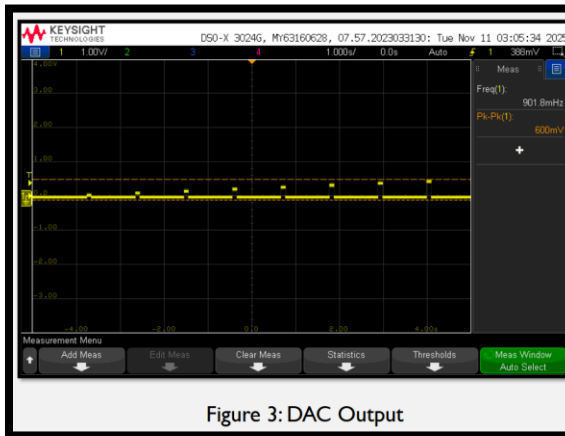


Figure 3: DAC Output

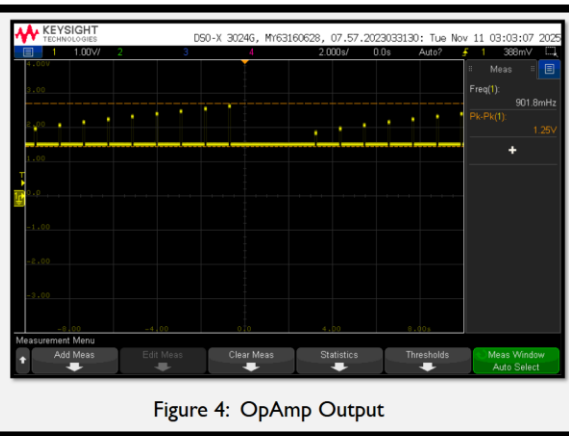


Figure 4: OpAmp Output

Figure 14: Lab 21 Data

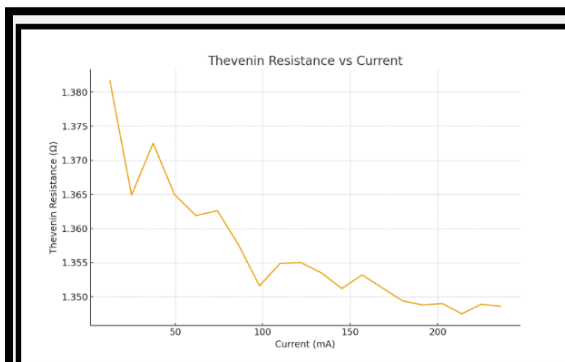


Figure 5: Arduino 5V
 $R_{\text{Thev}} \sim 1.35 \Omega$

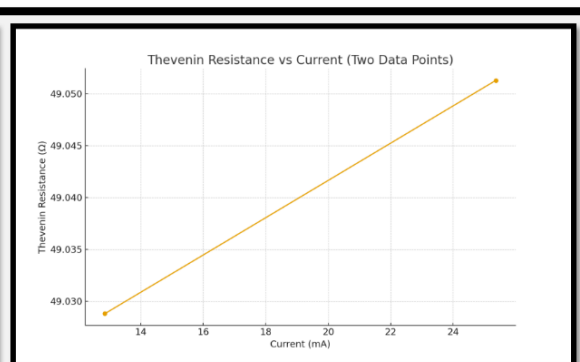


Figure 6: 5V DC Function Generator
 $R_{\text{Thev}} \sim 50 \Omega$

Figure 15: Lab 21 Data from the same Voltage Sources

Measurement Analysis:

There is completed board 4 instrument droid performs just as the breadboard prototype intended. The 300mA and 3A switch functions correctly as well. A load source greater than 300mA was not tested, which is why both sides of the switch give similar measurements. But the 3A switch does have a higher limit (assuming the code is adjusted correctly to $R_{\text{Sense}} = 1$).

Figure 13 only serves the verify the accuracy of the circuit. The waveform generator guarantees a Thevenin resistance of $50\ \Omega$, so if the board measures $50\ \Omega$ (which it does), then it is confirmed to be working.

5V Arduino Source:

- The completed board 4 PCB measures: $R_{\text{Thev}}: \sim 1.225\ \Omega$
- Lab 21's breadboard prototype measures $R_{\text{Thev}}: \sim 1.35\ \Omega$
- Roughly the same measurements for both. I believe the PCB board designed to be more accurate

5V Waveform:

- The completed board 4 PCB measures: $R_{\text{Thev}}: \sim 50\ \Omega$
- Lab 21's breadboard prototype measures $R_{\text{Thev}}: \sim 50\ \Omega$
- Roughly the same measurements for both

I believe these results determine my designed and assembled board 4 to be fully functional. It behaves exactly as expected.

Extra Features:

My board 4 features an OLED screen to display the active measurements, a buzzer that beeps with an increase pitch with each following measurement, and smart LEDs that signify the voltage droop. (The Arduino code is provided at the end)

What Worked:

1. The board entirely functions
2. The ATmega328 chip still works from board 3
3. The DAC/ADC circuit functions correctly
4. The MOSFET turns on
5. The sense resistor switch functions correctly
6. The board is compatible with a standard Arduino Uno R3
7. The OLED screen functions as intended displaying measurements
8. The buzzer functions as intended increasing in pitch with each measurement
9. The smart LEDs function as intended based off the V_{Load}/V_{Thev} droop ratio

Analysis of Project:

1. *What worked well and should be carried over to the next board?*
 - a. Clear labels made assembly of the board easy as it was simple to tell what component should go where. Separating sections of the board in the Altium schematic significantly organized the design of the circuit. Including multiple isolation switches made it clear what parts of the circuit were functioning correctly. Only uploading code in increments helped debug and ensure each peripheral was working as intended.
2. *What should be done differently?*
 - a. Although there were no issues, I think the Altium PCB wiring could be improved. I tried to make a more compact board, but it made it significantly harder to assemble. I did not spend enough time on organization leading to odd trace routes and a mirrored OLED header pin.
3. *Any hard errors?*
 - a. No hard errors occurred preventing function.
4. *Any soft errors that can be corrected for the next board?*
 - a. No specific errors occurred when recording data. But during the PCB design phase, I flipped the orientation of the OLED's header pins. So instead of hanging over the PCB, it has to be oriented the opposite way hanging away from the PCB.

Conclusion:

The custom “Golden Arduino” Board 3 was successfully designed and assembled. It correctly meets each function and design requirement allowing the board to act as an instrument droid capable of measuring load current, load voltage, Thevenin voltage, and Thevenin resistance while successfully operating an OLED screen, a buzzer, and 3 smart LEDs. After successfully debugging and updating code, functionality testing sketches were run to ensure correct operation. Multiple experiments were conducted and results were compared to results from our lab 21 prototype. The measured data shows this new board behaves exactly as intended compared to the verified prototype’s data. Most notably the calculated Thevenin resistance for the 5V Arduino source and 5V waveform are incredibly close when compared. The official board, however, does have one minor soft error regarding the orientation of its screen. Overall, this project broadened my experiment testing, creative design, and debugging skills. I learned how to implement multiple peripherals in one board in addition to its core function of collecting measurements. This final board allowed me to put all my gained knowledge from this course into one last design.

Arduino Code:

```

#include <Adafruit_MCP4725.h>
#include <Adafruit_ADS1X15.h>
#include <SFE_MicroOLED.h>
#include <Adafruit_NeoPixel.h>
#include <Wire.h>
#include <SPI.h>

#define PIN_RESET -1 //OLED reset tied to Arduino RESET
#define PIN_DC 6 //D/C wired to PWM 6
#define PIN_CS 10 //CS wired to PWM 10

#define LED_PIN 5
#define NUM_LEDS 3

#define BUZZER_PIN 9

//LEDs
Adafruit_NeoPixel strip(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);

//OLED
MicroOLED oled(PIN_RESET, PIN_DC, PIN_CS);

Adafruit_ADS1115 ads;
Adafruit_MCP4725 dac;
float R_sense = 10; //current sensor
long itime_on_msec = 100; //on time for taking measurements
long itime_off_msec = itime_on_msec * 10; // time to cool off
int iCounter_off = 0; // counter for number of samples off
int iCounter_on = 0; // counter for number of samples on
float v_divider = 5000.0 / 15000.0; // voltage divider on the VRM
float DAC_ADU_per_v = 4095.0 / 5.0; //conversion from volts to ADU
int V_DAC_ADU; // the value in ADU to output on the DAC
int I_DAC_ADU; // the current we want to output
float I_A = 0.0; //the current we want to output, in amps
long itime_stop_usec; // this is the stop time for each loop
float ADC_V_per_ADU = 0.125 * 1e-3; // the voltage of one bit on the gain of 1 scale
float V_VRM_on_v; // the value of the VRM voltage
float V_VRM_off_v; // the value of the VRM voltage
float I_sense_on_A; // the current through the sense resistor
float I_sense_off_A; // the current through the sense resistor
float I_max_A = 0.25; // max current to set for
int npts = 20; //number of points to measure
float I_step_A = I_max_A / npts; //step current change

float I_load_A; // the measured current load
float V_VRM_thevenin_v;
float V_VRM_loaded_v;
float R_thevenin; int i;

//For the buzzer:
//C major scale repeated across octaves
int scaleNotes[] = {
    262, 294, 330, 349, 392, 440, 494, // C4 D4 E4 F4 G4 A4 B4

```

```

    523, 587, 659, 698, 784, 880, 988,    // C5 D5 E5 F5 G5 A5 B5
    1047, 1175, 1319, 1397, 1568, 1760  // C6 D6 E6 F6 G6 A6
};

int NUM_SCALE_NOTES = sizeof(scaleNotes) / sizeof(scaleNotes[0]);

void setup() {
    Serial.begin(115200);

    // --- OLED init ---
    Wire.begin();           // not strictly needed for SPI, but safe
    oled.begin();           // init using default SPI port
    oled.clear(ALL);        // clear internal display RAM
    oled.display();         // show splash for a moment
    delay(500);
    oled.clear(PAGE);       // clear the screen buffer

    strip.begin();
    strip.show();           // all off
    strip.setBrightness(30); // 0-255

    dac.begin(0x60);        // address is either 0x60, 0x61, 0x62, 0x63, 0x64 or 0x65
    dac.setVoltage(0, false); //sets the output current to 0 initially
    // ads.setGain(GAIN_TWOTHIRDS); // 2/3x gain +/- 6.144V 1 bit = 3mV      0.1875mV (default)
    ads.setGain(GAIN_ONE);   // 1x gain   +/- 4.096V 1 bit = 2mV      0.125mV
    // ads.setGain(GAIN_TWO);   // 2x gain   +/- 2.048V 1 bit = 1mV      0.0625mV
    // ads.setGain(GAIN_FOUR);   // 4x gain   +/- 1.024V 1 bit = 0.5mV    0.03125mV
    // ads.setGain(GAIN_EIGHT);  // 8x gain   +/- 0.512V 1 bit = 0.25mV   0.015625mV
    // ads.setGain(GAIN_SIXTEEN); // 16x gain  +/- 0.256V 1 bit = 0.125mV  0.0078125mV
    ads.begin(0x48);         // note- you can put the address of the ADS111 here if needed
    ads.setDataRate(RATE_ADS1115_860SPS); // sets the ADS1115 for higher speed

    pinMode(BUZZER_PIN, OUTPUT);
    digitalWrite(BUZZER_PIN, LOW);
}

void loop() {
    for (i = 1; i <= npts; i++) {
        I_A = i * I_step_A;
        dac.setVoltage(0, false); //sets the output current
        func_meas_off();
        func_meas_on();
        dac.setVoltage(0, false); //sets the output current

        I_load_A = I_sense_on_A - I_sense_off_A; //load current
        V_VRM_thevenin_v = V_VRM_off_v;
        V_VRM_loaded_v = V_VRM_on_v;
        R_thevenin = (V_VRM_thevenin_v - V_VRM_loaded_v) / I_load_A;

        if (V_VRM_loaded_v < 0.75 * V_VRM_thevenin_v) i = npts; //stops the ramping

        Serial.print(i);
        Serial.print(", ");
        Serial.print(I_load_A * 1e3, 3);
        Serial.print(", ");
        Serial.print(V_VRM_thevenin_v, 4);
    }
}

```

```

Serial.print(", ");
Serial.print(V_VRM_loaded_v, 4);
Serial.print(", ");
Serial.println(R_thevenin, 4);

//Activates the screen
showOnOLED(i, I_load_A, V_VRM_thevenin_v, V_VRM_loaded_v, R_thevenin);

//Activates the buzzer
beepIndexTone(i); // Beep every index with rising pitch

//Activates the LEDs
updateLEDs(i, V_VRM_thevenin_v, V_VRM_loaded_v);

}

Serial.println("done"); delay(10000);
}

void func_meas_off() {
  dac.setVoltage(0, false); //sets the output current
  iCounter_off = 0; //starting the current counter
  V_VRM_off_v = 0.0; //initialize the VRM voltage averager
  I_sense_off_A = 0.0; // initialize the current averager
  itime_stop_usec = micros() + itime_off_msec * 1000; // stop time

  while (micros() <= itime_stop_usec) {
    V_VRM_off_v = ads.readADC_Differential_0_1() * ADC_V_per_ADU / v_divider + V_VRM_off_v;
    I_sense_off_A = ads.readADC_Differential_2_3() * ADC_V_per_ADU / R_sense + I_sense_off_A;
    iCounter_off++;
  }

  V_VRM_off_v = V_VRM_off_v / iCounter_off;
  I_sense_off_A = I_sense_off_A / iCounter_off;

  // Serial.print(iCounter_off); Serial.print(", ");
  // Serial.print(I_sense_off_A * 1e3, 4); Serial.print(", ");
  // Serial.println(V_VRM_off_v, 4);
}

void func_meas_on() {
  //now turn on the current

  I_DAC_ADU = I_A * R_sense * DAC_ADU_per_v;
  dac.setVoltage(I_DAC_ADU, false); //sets the output current

  iCounter_on = 0;
  V_VRM_on_v = 0.0; //initialize the VRM voltage averager
  I_sense_on_A = 0.00; // initialize the current averager
  itime_stop_usec = micros() + itime_on_msec * 1000; // stop time
  while (micros() <= itime_stop_usec) {
    V_VRM_on_v = ads.readADC_Differential_0_1() * ADC_V_per_ADU / v_divider + V_VRM_on_v;
    I_sense_on_A = ads.readADC_Differential_2_3() * ADC_V_per_ADU / R_sense + I_sense_on_A;
    iCounter_on++;
  }
}

```

```

    dac.setVoltage(0, false); //sets the output current to zero

    V_VRM_on_v = V_VRM_on_v / iCounter_on;
    I_sense_on_A = I_sense_on_A / iCounter_on;
    // Serial.print(iCounter_on);Serial.print(", ");
    // Serial.print(I_sense_on_A * 1e3, 4);Serial.print(", ");
    // Serial.println(V_VRM_on_v, 4);
}

void showOnOLED(int index, float I_load_A, float V_thev, float V_load, float R_thev) {
    oled.clear(PAGE);
    oled.setFontType(0);

    oled.setCursor(0, 0);
    oled.print("Idx:");
    oled.println(index);

    oled.setCursor(0, 9);
    oled.print("I:");
    oled.print(I_load_A * 1e3, 1);
    oled.println("mA");

    oled.setCursor(0, 18);
    oled.print("Vt:");
    oled.print(V_thev, 3);
    oled.println("V");

    oled.setCursor(0, 27);
    oled.print("Vl:");
    oled.print(V_load, 3);
    oled.println("V");

    oled.setCursor(0, 36);
    oled.print("R:");
    oled.print(R_thev, 2);
    oled.println("Ohm");

    oled.display();
}

void updateLEDs(int index, float V_thev, float V_load) {
    strip.clear();

    //Progress: 1-3 LEDs lit based on index
    int lit = map(index, 1, npts, 1, NUM_LEDS);
    if (lit < 1) lit = 1;
    if (lit > NUM_LEDS) lit = NUM_LEDS;

    //Droop health color
    float droop = V_load / V_thev; // ~1.0 = perfect
    uint32_t color;

    if (droop > 0.9) {
        color = strip.Color(0, 255, 0); // green = good
    } else if (droop > 0.8) {
        color = strip.Color(255, 150, 0); // orange = meh
    }
}

```

```

    } else {
        color = strip.Color(255, 0, 0); // red = bad
    }

    for (int j = 0; j < lit; j++) {
        strip.setPixelColor(j, color);
    }

    strip.show();
}

void beepBuzzer(int duration_ms) {
    digitalWrite(BUZZER_PIN, HIGH);
    delay(duration_ms);
    digitalWrite(BUZZER_PIN, LOW);
}

void beepIndexTone(int index) {
    int noteIndex = index - 1;

    //Clamp just in case
    if (noteIndex < 0) noteIndex = 0;
    if (noteIndex >= NUM_SCALE_NOTES) noteIndex = NUM_SCALE_NOTES - 1;

    int freq = scaleNotes[noteIndex];

    tone(BUZZER_PIN, freq); //Play musical note
    delay(70); // short note length
    noTone(BUZZER_PIN);
}

```